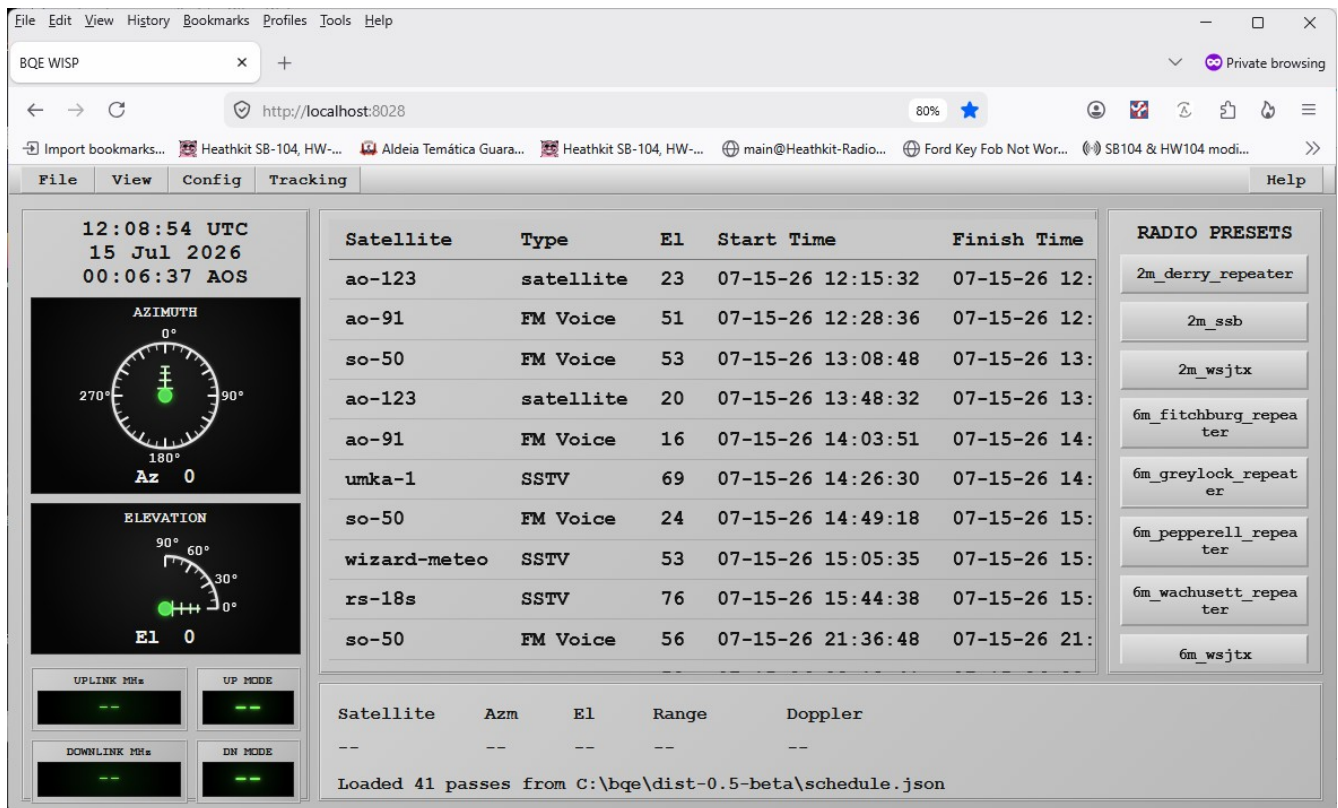


BQE-Wisp

LEO Satellite Scheduling, Tracking and Tuning software for Linux and Windows.



BQE Wisp is a Sunday afternoon automation project that got out of control, and over time grew into a from scratch re-implementation of the major functionality of Chris Jackson's Wisp32, which was the Gold Standard for satellite tracking throughout the 1990's, but which is no longer maintained.

In its current form, BQE-Wisp is a collection of scripts and data files that run on both Windows and Linux. Both a command line interface, and web UI are provided.

Acknowledgements

- My longstanding Gratitude and admiration to Chris Jackson, ZL2TPO for his original version all those years ago, which inspired me to pursue Satellites and Automation.
- Also, a major Thank You to Tony Thompson – G1RBO who has spent a significant effort to test BQE Wisp, as well as for his collaboration to support his own G1RBO radio control programs.

License

As the original Wisp software appears to be abandoned, I decided to write my own, and to make it freely available, with the stipulation that it not be used, modified or sold for commercial purposes. For this reason, I cannot call it “Open Source”, as true Open Source would apply to commercial activities as well. That said, I have attempted to be as permissive as possible for non-commercial end educational use. The exact license is included in the program folder as License.txt, as well as via the web UI as Help → License.

Other provisions:

- If you find and fix a but, or modify the software, please share the updates back to me.
- If you like the software and are using it, please drop me an email, or a digital qsl card – I like to know that folks are enjoying it.
- Please consider making a donation to the Amsat organization of your choice.

High level description of operation

BQE Wisp allows a user to define one or more low orbit amateur radio satellites in terms of their favorite frequencies and operating modes. These definitions are known as “nicknames”. The software will match these nicknames to orbital tracking data, and generate a schedule of times when the satellite(s) are visible.

As each satellite comes into view, the users radio will be automatically tuned to the uplink and downlink frequencies, which will be automatically updated to account for Doppler shift.

Between passes, BQE Wisp can optionally be configured with an Idle Task that will tune to the users radio to a specified “preset” (i.e. a channel) to do other things, such as monitor a repeater, or other frequencies of interest. BQE Wisp allows presets, and the “Idle Task” to run a program such as mmsstv (qsstv for Linux), or wsjtx etc. to allow for a wide variety of automated activities between passes.

BQE Wisp consists of 4 major python scripts intended to be executed by the user as well as several yaml configuration files, which will be explained later in this document.

Script	Arguments	Notes
bqe_download_keps.py	None	Downloads keps from several distribution sites to a file called keps.txt
bqe_schedule_passes.py	--nickname <nickname1> -- nickname <another_nickname> etc.	In cases where passes intersect, the pass with the highest maximum elevation will be scheduled. (Note – arguments are preceded by TWO dashes (minus signs.) --
bqe_wisp.py & bqe_wisp_web.py	None	The web console is viewable at localhost:8028

An additional script called **bqe_track_continuously.py** is called by **bqe_wisp** at pass time to perform tracking and tuning for the duration of the pass. This script can also be executed manually by itself via command line containing the necessary arguments, but there should be no need to do so. (Python **bqe_track_continuously.py** <nickname> (i.e. **bqe_track_continuously.py** iss)

Nicknames

Orbital Tracking data (Aka Keps), as downloaded from the internet, consist of a mathematical representation of a satellites orbit. They do not, however, contain any frequency or mode information. BQE_WISP uses the notion of a “nickname” to tie a specific satellite operating configuration to the orbital tracking data. This is done in a file called bqe_config/satellites.yaml. New satellites can be added as they are launched. Multiple operating modes may also be specified for satellites which switch modes (ISS). The only requirement is that nicknames be unique, and not contain whitespace or strange characters.

Presets are non-satellite “channel configurations” which can be called between passes to perform other automated activities. Channels also use the concept of a nickname to refer to the preset channels.

Presets are stored in a subfolder called presets. The naming convention is <nickname>_preset.yaml, where the nickname matches the nickname listed inside the file.

Note that the existing presets are based on my personal preferences, and assume U.S. frequency allocations. A user is free to define any presets they desire, subject to their countries frequency allocations, and the capabilities of the target radio. Unwanted presets can be and can be revived by removing the files from the presets folder, and restarting the program.

How to get started

Supported Platforms

Windows 10 & 11, as well as recent versions of Linux (Ubuntu etc.)

System Requirements

Recent Windows / Linux PC with Python3, and Hamlib installed and visible within the users path. Some python libraries may need to be installed, if they are not already present. A more complete discussion of the associated dependencies is included in a file called DEPENDENCIES.md, in the main program folder.

Initial configuration

Getting Started in Dos or Shell Command line mode

- Eventually, this code will be hosted on github where it can be cloned directly, but for now, create a folder somewhere (preferably at the top level of the directory structure) and unzip the distribution zip into this newly created folder.
- Edit **bqe_config/my_qth.yaml** to contain your Callsign, Latitude / Longitude etc, as well as an idle task if required. (See the existing data for an actual example.)
- Edit **bqe_config/my_rig.yaml** to select the proper radio type and port etc. (Note: Radio type “1” is a dummy test, and can be used to try the software without sending any radio commands.)
IMPORTANT NOTE: Currently bqewisp has only been tested with the Yaesu FT-736r and FT-991a. Other rigs supported by Hamlib will likely work. For FT-736r users who have an extra band module, adding an intermediate frequency in bqewisp/my_rig.yaml will allow bqewisp to switch between mode b and mode J satellites, in a manner

analagous to the “rev” button. Without a third band module, FT-736r users must be careful to not have U/V and V/U satellites in the same schedule.

- Edit **satellites.yaml** to add or update entries for any satellites you intend to use that are not already listed. This file is what ties a satellite name and catalog number to a nickname, and allows you to specify any extra information that the program may need. Fields are self explanatory (I hope) For now, Antenna Tracking is disabled, as the code refers to my own home brew antenna switch, and likely not of wide interest. The next release will support hamlib based rotators.

Getting Started in Browser mode

Unzip the code into a folder

Open a command window and cd into the folder.

Type `Python bq_e_wisp.py` You should see messages indicating that the server come up and start printing messages. Leave this window open. (A later release will make this a bit nicer.)

Once the server is up, open a browser and connect to <http://localhost:8028>

From there, select the Configuration menu bar item, and update both the rig and qth information.

After this has been updated, restart the server, either by using “File/Exit” in the menu bar, or simply hitting ctrl-c in the command window.

Once the server has been shut down, restart it by again typing `python bq_e_wisp.py`

Enter the Web console again and execute the Tracking/Update Keps menu item, followed by Tracking/ Autoschedule passes.

How to run

Note: FT-736r users should make sure the satellite switch is set to “off” on the radio before running to allow for proper idle time switching to non-satellite modes between passes.

After the initial configuration described above, to run the the program, simply open a command window and cd into the working directory and type the following:

- **`python bq_e_update_keps.py`**
- **`python bq_e_schedule_passes.py --nickname iss --nickname umka-1` etc. Note that the arguments are Nicknames which must have corresponding entries in “satellies.yaml”. If the**

same set of satellites will often be scheduled, the `--auto_schedule` flag may be used to schedule many satellites at once. Auto_schedule will attempt to schedule all satellites that have the `auto_schedule: True` flag set in `satellites.yaml`. Downloading of keps, and auto-scheduling can also be performed from within the UI, once the program has been initially configured and started. (A restart may be required if the QTH has changed.)

-

```
nickname: "umka-1"
satellite_name: "UmKA 1 (RS40S)"
satellite_type: "SSTV"
catalog_number: '57172'
satellite_mode_required: True
downlink_frequency_mhz: '437.625'
downlink_mode: 'FM'
uplink_frequency_mhz: '145.850'
uplink_mode: 'FM'
bandwidth: '16000'
ctcss_tone: '0'
enable_antenna_tracking: False
enable_tuning: True
priority: '3'
auto_schedule: True
program_to_run_during_pass: "C:/Ham/MMSSTV/MMSSTV.EXE"
```

- Satellite schedules often overlap. In the event of conflicting passes, the satellite with the highest pass maximum pass elevation will be selected. (At some future point, I hope to implement a more advanced priority system, similar to what the original WISP had.)

- **python bqe_wisp.py**

Once started, **bqe_wisp.py** will load the schedule data created in the previous steps, and run through the schedule until all passes have been executed. For each pass, **bqe_wisp.py** will launch **bqe_track_continuously.py** to perform the tracking and tuning actions. Progress info is output to the command window, however, a more detailed UI is available via a browser window at <http://localhost:8028>.

As each pass progresses, tracking and tuning data is calculated every 30 seconds when the satellite is at low elevation, as Doppler and position do not change much. During times when the satellite rises above 65 degrees of elevation, the program will automatically switch to perform tracking and tuning actions every 15 seconds to better keep up with the changing frequency and position, while attempting to minimize start/stop wear on any az/el rotator systems.

The program can be terminated at any time by hitting ctrl-c, and can be restarted cleanly using the existing schedule, even if some passes are already in the past.

When a pass, or idle time interval ends, any companion program started by bqe_wisp will be stopped as well.

Additional notes

The Original WISP supported the Kansas City Tracker (An ISA card from the 1990's) as well as supplying tracking info by Windows DDE. Both methods appear to be totally deprecated at this point.

The Antenna tracking supplied with this version of bqe-wisp is currently tied to my personal switched array (i.e. No moving parts) antenna controller, and is likely not of use to anyone other than myself. I will be moving this functionality out of bqe-wisp, and replacing it with hamlib rotctld support in the next couple of weeks.

Presets:

As described earlier, presets are used to define configurations for the radio to use when not actively tracking satellites. Both at program startup, and after each path, the idle task will tune to the preset name listed in my_qth.yaml. Other presets can be dynamically selected via the "View → Show Presets" menu bar item, which will expose a panel containing a list of configured presets. These can be used as memory channels to tune around while waiting, if a temporary alternative to the idle task is desired. . After the next pass, the idle task preset will again be run. Preset display is off by default, and must be enabled from the menu bar when desired.



Known Issues

- Currently all satellites need an `uplink_frequency` defined in `satellites.yaml` for the program to work correctly. In cases of receive only satellites (umka-1 and other sstv satellites) where no uplink exists, simply inventing one in a different band is fine. I'll be working on a better fix shortly.
- Same band/frequency satellites (Such as the APRS digipeater) will not work with an FT-736r, due to a hardware limitation in the radio.

Support

Best effort support is available via wb1bqe@gmail.com. Suggestions/feedback are quite welcome as well. After any kinks are worked out, my intention is to release this via Github as a public source offering, at which time, contributions via pull requests will also be welcomed.

Tips

- When using `bqe_wisp` with the FT-736r, the “Sat Mode” switch must be off. (The software will enable `satellite_mode` automatically.)
- When using `bqe_wisp` on a system where ‘python’ is not available, but `python3` is, please either create an alias between `python` and `python3`, or update the `python_path` setting in `bqe_config/general_settings.yaml`, as it is needed for some of the scripts to call others during program operation.

```
- common:
  | python_path: python    # Can be a full path
```